CSCE 3301 – Computer Architecture
Fall 2026
Project 1: femtoRV32

RISC-V Processor Design and Implementation Report

Asmaa Elshabshiri
900243596
Janna Alfiky
900241379

Introduction

The objective of this project was to design, implement, and test a RISC-V processor using Verilog HDL. The processor supports a range of RISC-V instructions, including arithmetic, logical, memory access, branching, jumping, and special instructions.

The main goal was to develop a working datapath and control unit capable of decoding instructions, performing the required operations, accessing memory, and updating the program counter correctly.

The processor was tested using a Verilog testbench that checks the functionality of the supported instructions and verifies the expected register, memory, and program counter values.

Processor Design

The processor uses a single-cycle datapath, where each instruction is processed within one clock cycle. The top-level module, Processor, connects all the components together.

The main components of the processor are:

- Register: Stores the current program counter.
- InstMem: Stores instructions and provides the instruction corresponding to the current PC.
- ControlUnit: Decodes instructions and generates control signals.
- RegFile: Stores the processor's 32 registers.
- rv32_ImmGen: Generates the immediate value according to the instruction format.
- TwoBy1Mux: Selects between different datapath inputs.
- ALUControl: Determines which ALU operation should be performed.
- prv32_ALU: Performs arithmetic, logical, shift, and comparison operations.
- DataMem: Handles memory read and write operations.
- BranchUnit: Determines the next program counter selection for branches and jumps.

The components are connected through internal wires declared in the Processor module. These wires carry instruction data, register values, immediate values, ALU results, and control signals.

Program Counter and Instruction Fetch

The processor begins execution using the program counter stored in the Register module. The current program counter is stored in pcout, while pcin represents the next address to be loaded.

The halt signal controls whether the PC is updated. When halt is zero, the load signal becomes one, allowing the PC to update on the next clock edge.

The instruction memory receives the current PC. The InstMem module contains an array of 64 32-bit instructions. It uses bits [7:2] of the address to select the instruction.

The next sequential instruction address is calculated by adding four to the current PC. This is used when the processor executes instructions that do not change the normal execution flow.

Instruction Decode and Control Unit

The ControlUnit module receives three fields from the fetched instruction:

- Opcode
- funct3
- Instruction bit 20 (inst20)

Based on the opcode, the control unit generates the signals required to execute the instruction.

At the beginning of its combinational block, the control unit initializes all control signals to default values. This ensures that signals not required by a particular instruction remain inactive.

The main control signals are:

- branch: Indicates a conditional branch instruction.
- jal: Indicates a JAL instruction.
- jalr: Indicates a JALR instruction.
- lui: Identifies the LUI instruction.
- memRead: Enables reading from data memory.
- memWrite: Enables writing to data memory.
- regWrite: Enables writing to the register file.
- ALUASel: Selects the first ALU operand.
- ALUBSel: Selects the second ALU operand.
- WBSel: Controls the write-back multiplexer selection.
- ALUOp: Indicates the general ALU operation category.
- Halt: Prevents the PC from updating when asserted.

The control unit handles each instruction category differently.

R-type instructions

For R-type instructions, register writing is enabled and ALUOp is set to ALUOP_R. Both ALU operands are taken from the register file.

I-type arithmetic instructions

For I-type instructions, register writing is enabled, the second ALU input is selected from the immediate generator, and ALUOp is set to ALUOP_I.

Load instructions

For loads, regWrite and memRead are enabled. The ALU uses the base register and immediate value to calculate the memory address. WBSel is set to WB_MEM so the loaded data is written into the destination register.

Store instructions

For stores, memWrite is enabled and the immediate is selected as the second ALU operand. The ALU calculates the memory address using the base register and immediate.

Branch instructions

For branches, the branch signal is asserted and ALUOp is set to ALUOP_SUB. The ALU subtracts the two source registers to generate the condition flags used by the branch unit.

JAL instructions

For JAL, the jal signal is asserted, register writing is enabled, and WBSel is set to WB_PC4. This allows the return address, PC + 4, to be written into the destination register.

JALR instructions

For JALR, the jalr signal is asserted, register writing is enabled, and the second ALU input is selected from the immediate generator. The write-back selection is also set to WB_PC4.

LUI instructions

For LUI, register writing is enabled, the second ALU input is selected from the immediate generator, and the lui signal is asserted.

The lui signal is important because it is passed to the ALU control module. It causes the ALU to select the pass operation, allowing the immediate value to be written directly into the destination register without adding it to another operand.

AUIPC instructions

For AUIPC, register writing is enabled, the first ALU input is selected from the current program counter, and the second input is selected from the immediate generator. This allows the ALU to calculate the sum of the current PC and the upper immediate value.

System instructions

For system instructions, the control unit checks funct3 and instruction bit 20.

ECALL is treated as a no-operation instruction, while EBREAK asserts halt and stops the PC from updating.

Other instructions not explicitly handled by the control unit, including the supported FENCE-related instructions, retain the default control signals and behave as no-operations.

ALU Control

The ALUControl module receives ALUOp, funct3, instruction bit 30 (inst30), and the lui signal. It generates the four-bit ALUSel signal that determines the exact operation performed by the ALU.

The implementation uses the following general ALU operation categories:

- ALUOP_ADD: Selects addition.
- ALUOP_SUB: Selects subtraction.
- ALUOP_PASS: Selects the pass operation.
- ALUOP_R: Decodes R-type operations using funct3 and inst30.
- ALUOP_I: Decodes I-type operations using funct3 and inst30.

For R-type instructions, the funct3 field determines the operation. For ADD and SUB, instruction bit 30 distinguishes between the two operations. Similarly, bit 30 distinguishes between SRL and SRA.

For I-type instructions, the same funct3 decoding is used. However, the ADDI instruction always selects addition, since its immediate value is not used to distinguish between addition and subtraction.

The module also handles LUI separately. When lui is asserted, ALUSel is overridden to select ALU_PASS, regardless of the operation selected earlier in the decoding process.

This is important because LUI does not need to perform an arithmetic operation. It simply passes the generated immediate value through the ALU to the write-back path.

ALU

The prv32_ALU module performs the actual arithmetic and logical operations.

Its two main inputs are ALUinput1 and ALUinput2, selected by the multiplexers in the top-level module. The operation is determined by ALUSel.

The supported operations include:

- ADD: Adds the two operands.
- SUB: Subtracts the second operand from the first.
- SLL: Performs a logical left shift.
- SLT: Performs a signed less-than comparison.
- SLTU: Performs an unsigned less-than comparison.
- XOR: Performs bitwise exclusive OR.
- SRL: Performs a logical right shift.
- SRA: Performs an arithmetic right shift.
- OR: Performs bitwise OR.
- AND: Performs bitwise AND.
- PASS: Passes the selected operand through the ALU.

The shift amount is taken from the five least significant bits of the second ALU input
The ALU also generates four flags: carry (cf), zero (zf), overflow (vf), and sign (sf). These flags are connected to the branch unit and are used when evaluating conditional branch instructions.

Branch Unit

The BranchUnit module determines which address should be loaded into the program counter.

It receives the branch, jal, and jalr control signals, along with funct3 and the four ALU flags.

For conditional branches, the module first determines whether the branch condition is satisfied.

The implemented conditions are:

- BEQ: zf
- BNE: ~zf
- BLT: sf != vf
- BGE: sf == vf
- BLTU: ~cf
- BGEU: cf

For signed comparisons, the sign and overflow flags are used together. For unsigned comparisons, the carry flag is used.

After determining the branch condition, the module selects the next PC source.

The selection follows this priority:

1. JALR: Selects the JALR target.
2. JAL: Selects the immediate-based target address.
3. Taken branch: Selects the branch target.
4. Otherwise: Selects PC + 4.

The output is the two-bit PCSel signal, which controls the two PC selection multiplexers in the top-level processor.

Register File

The register file is implemented using the RegFile module, which contains 32 registers, each 32 bits wide.

Instead of using a single memory array with a write enable, the implementation uses a generate loop to instantiate 32 separate Register modules. Each register receives the same input data and clock, but has an individual load signal.

The RegFile module generates a one-hot load vector. Initially, all load signals are set to zero. If register writing is enabled and the destination register is not x0, the corresponding load bit is set to one. This prevents writes to register x0. The two read ports are implemented using combinational assignments.

The individual Register module is built using a 32-bit multiplexer and 32 D flip-flops. For each bit, the multiplexer selects either the current stored value or the new input value, depending on the load signal. The selected value is then connected to the D flip-flop. This allows the register to retain its previous value when its load signal is inactive and store new data when its load signal is asserted.

The reset signal is connected to the D flip-flops to initialize the register storage.

Data Memory

The DataMem module implements a memory containing 64 words, with each word being 32 bits wide. The memory uses the address to determine both the word location and the byte position within that word.

The address bits [7:2] select one of the 64 memory words, while bits [1:0] determine the byte offset within the selected word.

The module supports three write operations:

- SB: Writes one byte into the selected byte position.
- SH: Writes two bytes into the selected halfword position.
- SW: Writes a complete 32-bit word.

Memory writes are synchronous and occur on the positive clock edge when MemWrite is asserted. Memory reads are combinational and occur when MemRead is asserted.

The module supports five load operations:

- LB: Loads one byte and sign-extends it to 32 bits.
- LH: Loads one halfword and sign-extends it to 32 bits.
- LW: Loads a complete 32-bit word.
- LBU: Loads one byte and zero-extends it to 32 bits.
- LHU: Loads one halfword and zero-extends it to 32 bits.

The funct3 field determines which memory operation is performed, while the byte offset determines the selected portion of the memory word.

Instruction Memory

The InstMem module contains an array of 64 32-bit instructions. It uses the current PC to select the instruction. Instruction retrieval is combinational, meaning that the instruction output changes when the address changes. The module is connected directly to the program counter through the Processor top-level module.

Write-Back Datapath

The write-back section determines which value is sent to the destination register. The implementation uses two multiplexers controlled by the two bits of WBSel.

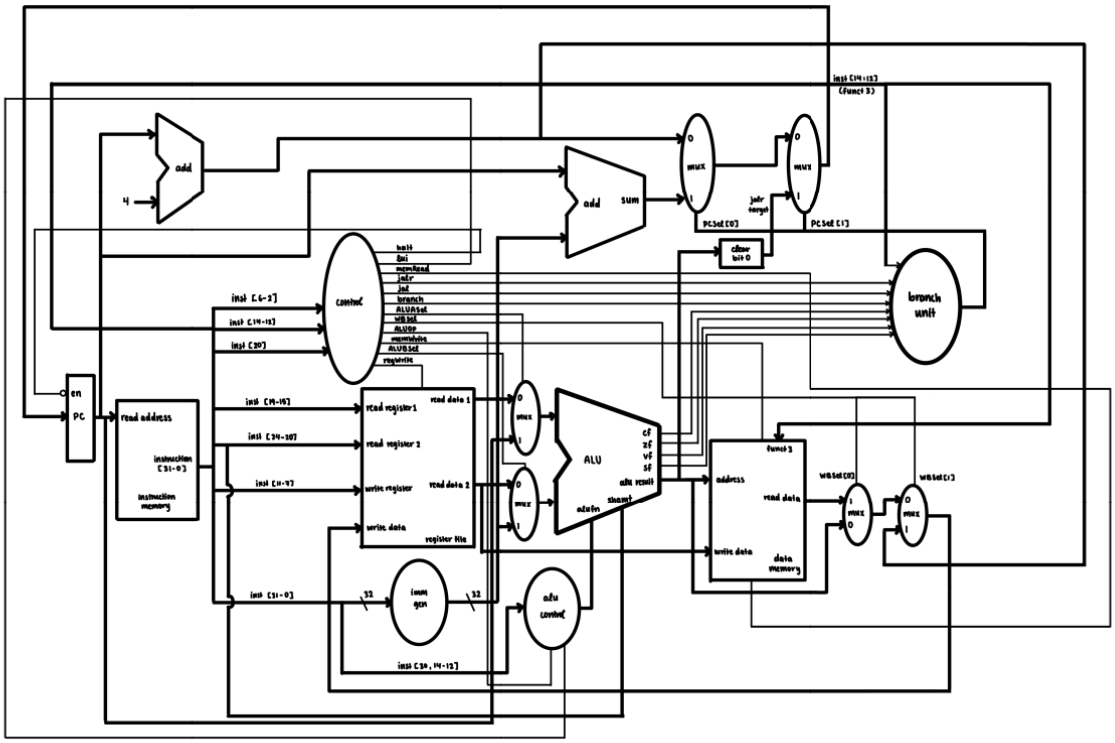
The first multiplexer selects between the ALU result and data memory output. The second selects between that result and pcplus4.

The available write-back sources are therefore:

- ALU result.
- Data memory output.
- Return address (PC + 4).

This allows arithmetic, logical, load, JAL, and JALR instructions to write their results through the same register-file input.

Complete Datapath



Top-Level Processor Implementation

The top-level module is named Processor and is responsible for connecting all the modules described above. It declares the internal wires required to transfer data and control signals between the components.

The implementation can be divided into five main sections:

1. Instruction fetch: The PC register and instruction memory retrieve the current instruction.
2. Instruction decoding: The control unit generates the signals required by the instruction, while the register file and immediate generator provide the operands.
3. Execution: The ALU control selects the operation, and the ALU performs the required calculation.
4. Memory and write-back: The data memory handles load and store operations, and the write-back multiplexers select the result to be written into the register file.
5. Next PC calculation: The branch unit and PC multiplexers determine the address of the next instruction.

The separation of the processor into individual modules makes the implementation easier to organize and allows each component to be tested independently.

Supported Instructions

The processor was implemented to support the following instruction categories:

- R-type: ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
- I-type: ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
- Load: LB, LH, LW, LBU, LHU
- Store: SB, SH, SW
- Branch: BEQ, BNE, BLT, BGE, BLTU, BGEU
- Jump: JAL, JALR
- U-type: LUI, AUIPC
- Special: ECALL, EBREAK, FENCE, FENCE.TSO, PAUSE

The supported instructions use the same datapath components but different control signals depending on their instruction formats.

For example, arithmetic instructions use the ALU result directly, load instructions select data memory output during write-back, and jump instructions select PC + 4 as the value written into the destination register.

LUI uses the ALU pass operation to write the immediate value, while AUIPC uses the ALU to add the immediate value to the current program counter.

Testing and Verification

Testbench Design

A separate Verilog testbench was developed to verify the processor's functionality. The testbench initializes the clock and reset signals and provides instructions directly to instruction memory. It then checks the processor's outputs after each instruction is executed.

Several tasks were created to simplify the testing process:

- reset_cpu: Resets the processor before each instruction test.
- check_reg: Tests arithmetic and logical instructions by comparing the destination register against the expected result.
- check_load: Tests load instructions by checking the value written into the destination register.
- check_store: Tests store instructions by checking the updated memory contents.
- check_branch: Tests branch instructions by checking the resulting program counter.

For register-based tests, the testbench forces the source registers to known values so that the expected results can be calculated easily.

For load tests, data memory is initialized with a predefined word. For store tests, the memory is initialized with a known value before executing the instruction.

JAL, JALR, LUI, AUIPC, and the special instructions are tested separately.

The testbench uses hierarchical references to inspect internal processor signals, including the register file, instruction memory, data memory, and program counter.

Test Coverage

The testbench contains 42 individual checks distributed across the supported instruction categories.

The tests verify different aspects of the processor:

- Arithmetic and logical results are checked through the destination register.
- Load operations are checked through the destination register.
- Store operations are checked by inspecting data memory.
- Branch instructions are checked by inspecting the program counter.
- Jump instructions are checked through both the destination register and program counter.
- Special instructions are checked by observing the program counter behavior.

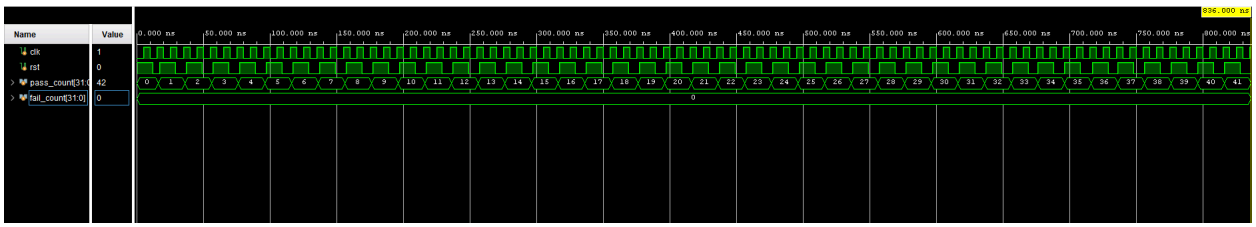
Simulation Results

The processor was simulated using the developed testbench. Each instruction was executed separately, and its actual output was compared against the expected result.

The final simulation output was:
MS2 SUMMARY: 42 PASS, 0 FAIL
ALL TESTS PASSED

All 42 tests passed successfully. This indicates that the processor produced the expected results for all the instructions included in the testbench.

Simulation Waveform



Conclusion

This project involved designing, implementing, and testing a RISC-V processor using Verilog HDL. The processor was built using separate modules for instruction memory, the program counter, register file, immediate generator, control unit, ALU control, ALU, data memory, and branch unit. These components were connected through the top-level Processor module to form the complete datapath.

The implementation supports arithmetic and logical operations, load and store instructions, conditional branches, jumps, U-type instructions, and selected special instructions. During testing, an issue with the AUIPC instruction was identified and corrected by changing the ALU input multiplexer to use the current program counter instead of the next program counter. After this correction, the processor successfully passed all 42 testbench checks, with zero failures.

Overall, the project provided practical experience in processor architecture, datapath design, control signal generation, Verilog implementation, simulation, and debugging. It also helped demonstrate how the different processor components work together to execute instructions.